

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: «Поиск подстроки в строке (КМП)»

Студент гр. 4343

Стариков Н.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P ,
- Вторая строка — T .

Выход: индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход:

- Первая строка — A ,
- Вторая строка — B .

Выход:

Если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе вывести -1. Если возможно несколько сдвигов, вывести первый индекс.

Описание алгоритмов и оценка их сложности во времени и памяти

В первой задаче реализуется алгоритм Кнута-Морриса-Пратта (КМП) для поиска всех вхождений шаблона P в текст T . Длина шаблона $n = |P| \leq 25000$, длина текста $m = |T| \leq 5000000$.

Алгоритм состоит из двух этапов: сначала вычисляется префикс-функция для шаблона P , затем выполняется проход по тексту с использованием префикс-функции для эффективного сдвига при несовпадении. Префикс-функция $\pi[i]$ для каждого i от 0 до $n-1$ определяется как длина наибольшего собственного префикса подстроки $P[0..i]$, совпадающего с её суффиксом. Вычисление префикс-функции выполняется за линейное время $O(n)$: для каждого i от 1 до $n-1$ используется значение $\pi[i-1]$ и производится переход по уже вычисленным значениям π , при этом каждый символ шаблона сравнивается не более одного раза. Затем поиск в тексте также выполняется за $O(m)$: указатель j в шаблоне обновляется по префикс-функции при несовпадении, а при полном совпадении ($j = n$) фиксируется позиция вхождения и выполняется откат $j = \pi[j-1]$ для возможности перекрывающихся вхождений.

Таким образом, общая временная сложность алгоритма составляет $O(n + m)$, что оптимально для данной задачи.

По памяти требуется $O(n)$ для хранения массива префикс-функции и ответов (в худшем случае до $O(m)$ для хранения всех индексов вхождений, но обычно ответы выводятся по мере нахождения). В реализации массив ответов может быть размещён динамически, и его размер не превышает m/n , но в худшем случае может быть порядка m (например, шаблон из одного символа), поэтому оценка памяти в наихудшем сценарии $O(m)$. Однако с учётом ограничений $m \leq 5 \cdot 10^6$ это приемлемо. Основной объём дополнительной памяти – массив π размера n .

Для второй задачи, в которой требуется определить, является ли строка A циклическим сдвигом строки B (при условии равенства длин), используется

алгоритм Кнута-Морриса-Пратта (КМП) без явного построения конкатенированной строки $A+A$. Сначала проверяется, что длины A и B совпадают; если нет, сразу возвращается -1. Далее для строки B (длиной n) вычисляется префикс-функция π за время $O(n)$ стандартным способом. Затем выполняется поиск вхождения строки B в виртуальной строке, образованной двумя последовательными копиями A . Для этого в цикле по i от 0 до $2n-1$ символ из A выбирается по индексу $i \bmod n$, то есть $\text{sig} = A[i \% n]$. Состояние автомата (текущая длина совпавшего префикса j) обновляется по правилам КМП с использованием префикс-функции π . Когда j достигает n (найденно полное вхождение), вычисляется позиция $\text{pos} = i - n + 1$. Если $\text{pos} < n$, то это означает, что вхождение начинается в первой копии A , и pos является искомым индексом начала строки B в A (циклический сдвиг). В случае, когда возможны несколько сдвигов (например, для периодических строк), алгоритм останавливается на первом найденном вхождении, удовлетворяющем условию $\text{pos} < n$, что соответствует требованию вывода первого индекса. Если после полного обхода $2n$ символов ни одного подходящего вхождения не найдено, возвращается -1.

По времени алгоритм работает за $O(n)$: вычисление префикс-функции занимает $O(n)$, поиск по виртуальной строке длины $2n$ также выполняется за $O(n)$, так как каждый символ A обрабатывается не более двух раз, а суммарное количество переходов по π линейно.

По памяти требуется $O(n)$ для хранения массива π и строк A , B ; дополнительная строка $A+A$ не создаётся, что оптимизирует потребление памяти. Итоговая сложность: время $O(n)$, память $O(n)$.

Тестирование

В рамках тестирования разработанных программ был создан набор unit-тестов на основе фреймворка Google Test. Для задачи поиска всех вхождений шаблона в текст (алгоритм КМП) были проверены следующие случаи:

Тест 1 (SimpleOccurrence) для шаблона "ab" и текста "abab" дал ожидаемые индексы 0 и 2;

Тест 2 (NoOccurrence) с шаблоном "abc" и текстом "abab" вернул пустой массив, что соответствует отсутствию вхождений;

Тест 3 (OverlappingOccurrences) для шаблона "aaa" и текста "aaaaa" правильно выявил позиции 0,1,2, что подтверждает способность алгоритма учитывать перекрывающиеся вхождения;

Тест 4 (EmptyPattern) для пустого шаблона вернул пустой массив (согласно условию, при пустом шаблоне выводится -1);

Тест 5 (PatternEqualsText) для шаблона "abc" и текста "abc" дал индекс 0;

Тест 6 (SingleCharacterPattern) для шаблона "a" в тексте "banana" возвратил индексы 1,3,5. Все эти тесты успешно пройдены.

Для задачи определения циклического сдвига (вторая задача) были реализованы следующие тесты:

Тест 1 (ExampleFromTask) для A="defabc", B="abcdef" выдал индекс 3, что совпадает с примером из условия;

Тест 2 (NoShiftDifferentLength) для строк разной длины ("abc" и "abcd") вернул -1;

Тест 3 (EqualStrings) для одинаковых строк "abc" и "abc" дал индекс 0;

Тест 4 (ShiftByOne) для A="cab", B="abc" вычислил индекс 2;

Тест 5 (PeriodicStringMultipleShifts) для A="abab", B="abab" вернул 0, так как требуется первый возможный индекс;

Тест 6 (EmptyStrings) для двух пустых строк вернул 0;

Тест 7 (OneEmpty) для A="a", B="" – возврат -1. Все тесты успешно пройдены, что подтверждает корректную работу программ.

Вывод

В ходе выполнения лабораторной работы были успешно реализованы два алгоритма на основе метода Кнута-Морриса-Пратта (КМП) для решения задач поиска подстроки и проверки циклического сдвига. Первая программа находит все вхождения шаблона P в текст T с линейной временной сложностью $O(|T|+|P|)$ и памятью $O(|P|)$ плюс хранение ответов. Алгоритм корректно обрабатывает перекрывающиеся вхождения, пустые строки и случаи, когда шаблон длиннее текста. Вторая программа определяет, является ли строка A циклическим сдвигом строки B , используя виртуальную удвоенную строку $A+A$ без явного выделения памяти, что позволяет достичь сложности $O(|A|)$ по времени и $O(|A|)$ по памяти. В реализации применён доступ к символам A по модулю индекса, что исключает избыточное конкатенирование.

Для обеих задач проведено автоматизированное тестирование с использованием Google Test. Набор тестов для первой задачи включает проверку простого и перекрывающегося вхождений, отсутствия вхождений, обработки пустого шаблона и шаблона из одного символа. Для второй задачи проверены пример из условия, строки разной длины, одинаковые строки, сдвиг на один символ, периодические строки, а также пустые и полупустые строки. Все тесты успешно пройдены, что подтверждает корректность работы алгоритмов в широком диапазоне входных данных, включая граничные условия.